

Of course! Here's the improved version of the **Starter Mini-Challenge** rewritten as a **clear, structured document**—now including your new ideas like adding/removing carts and enhanced history. It follows the same style as the original plan.

Dart Console Project: Advanced Mergeable Shopping Cart

Project Goal

Build a Dart console application that allows managing **multiple shopping carts**, each holding a list of items and their quantities. The app supports:

- Adding and removing carts
 - Merging carts
 - Viewing differences between carts
 - Tracking change history per cart
 - Undoing changes
-

Feature Set Overview (Step-by-Step Tasks)

Step 1: Define the Cart Data Structure

What to do:

- Each cart will store item names (`String`) and quantities (`int`).
- Maintain **multiple carts** in a central storage structure.
- Each cart is identified by a **unique name or ID**.

Data structure examples (conceptual):

- A single cart:
`"cartA" → {"apple": 2, "banana": 3}`
- All carts:
`Map<String, Map<String, int>>`

What you'll practice:

- Working with nested maps
 - Structuring data in scalable ways
-

+ Step 2: Add and Remove Carts ★ (Your enhancement)

What to do:

- Let the user **create a new cart** with a chosen name.
- Let the user **delete an existing cart** by name.
- Validate:
 - No duplicate names when creating
 - Cart must exist to delete it

Example (conceptual):

- Add: "cartHoliday" → empty cart
- Remove: "cartA" → remove from global map

What you'll practice:

- Adding/removing keys from maps
 - Input validation and user flow control
-

Step 3: Merge One Cart Into Another

What to do:

- Allow the user to select two existing carts:
 - **Source cart**: data that will be added
 - **Target cart**: will receive merged contents
- Items with matching names: sum their quantities
- Items only in source: add them to target
- Before merging, save a **copy of the target cart** to its history (see Step 5)

What you'll practice:

- Iterating and merging map data
- Safe data mutation
- Working with nested structures



Step 4: Compare Two Carts (Diff Viewer)

What to do:

- Let the user pick any two carts to compare
- Identify:
 - Items **added**
 - Items **removed**
 - Items with **changed quantities**
- Display the results clearly

Example (conceptual):

Cart A:

```
{"apple": 2, "banana": 3}
```

Cart B:

```
{"banana": 5, "orange": 1}
```

Diff:

- Added: "orange"
- Removed: "apple"
- Changed: "banana" → 3 → 5"

What you'll practice:

- Comparing map keys and values
- Building a difference report



Step 5: Implement Per-Cart Change History ★ (Your enhancement)

What to do:

- Every cart gets its own history list:

```
Map<String, List<Map<String, int>>>
```

- Before each destructive action (e.g. merge or delete), **push a snapshot** of the cart to its history
- Allow optional viewing of this history (e.g., with version numbers or timestamps)

What you'll practice:

- Working with nested data: list of maps inside a map
 - Designing rollback-friendly systems
-

Step 6: Undo Last Change for a Cart

What to do:

- Let user pick a cart and undo the most recent change
- If the cart's history has previous versions, restore the last one
- If no history is available, show a friendly message

What you'll practice:

- Safe rollback logic
 - Guarding against errors like empty lists
 - State restoration
-

Step 7: Create a Console Menu Interface

What to do:

Create a clean looped menu to let the user choose:

1. View all carts
2. Add new cart
3. Remove a cart
4. View a cart's content
5. Merge carts
6. Compare two carts
7. Undo last change on a cart

8. View cart history
9. Exit

Use `stdin.readLineSync()` to receive input and match it to functions.

What you'll practice:

- Writing user input logic
 - Building interactive console UIs
 - Handling invalid inputs
-



Step 8: Test Your Application

What to do:

Manually test the app by:

- Creating and deleting carts
- Merging with various overlap patterns
- Undoing merges
- Comparing identical or empty carts
- Viewing histories

What you'll practice:

- Manual testing flow
 - Thinking like a user
 - Debugging logic issues
-



Bonus Challenges (For After Core Features)

- Add item **price** and **category**
 - Allow editing specific items (change quantity or remove it)
 - Automatically generate sample carts for testing
 - Save/load all data from files (using JSON)
 - Track cart metadata like last updated time or owner
-

Execution Plan

Step	Description	Your Goal
1	Build core cart data structure	Set up the central storage
2	Add/remove carts	Build basic cart management
3	Merge carts	Practice map merging logic
4	Show differences between carts	Build a diff comparison tool
5	History per cart	Store versions of carts safely
6	Undo system	Enable rollback functionality
7	User interface	Make it interactive
8	Manual testing	Test all flows and bugs

Would you like to begin now with **Step 1: Building the cart structure and central storage?**
I can walk you through what to implement and what to think about before coding.